

AI-Powered Network Intrusion Detection System (AI-IDS)

Prepared By:

Hamza Zulfikar (ID: F2023376057)

Zafra Noor Ejaz (ID: F2023376056)

Date: July 2026**Environment:** Ubuntu Linux**Framework:** PyTorch & MLOps Suite

1. PROJECT OVERVIEW

In the modern interconnected digital ecosystem, network security has transformed from a perimeter defense strategy into an evolving adversarial challenge. Traditional signature-based systems suffer severe latency in adapting to day-zero vulnerabilities. This project presents a sophisticated, production-ready **AI-based Intrusion Detection System (AI-IDS)** engineered leveraging deep learning architectures to classify high-velocity network traffic into standard operational metrics or malicious vectors.

By treating network analysis as a granular classification challenge, the proposed system ingests high-dimensional behavioral attributes to determine anomalies dynamically. Beyond structural model development, this architecture is completely operationalized via a rigorous Machine Learning Operations (MLOps) cycle, embedding automated lifecycle versioning, asynchronous deployment pipelines, and high-availability orchestration mechanisms to match modern corporate cloud infrastructure standardizations.

2. PROBLEM STATEMENT

Legacy intrusion detection frameworks depend heavily on predefined database rulesets. While effective for well-documented historical attack definitions, they exhibit fatal vulnerabilities when confronted with polymorphic malware, obfuscated execution chains, and distributed denial-of-service (DDoS) campaigns that mimics legitimate burst traffic.

The objective of this design is to alleviate rule dependency by constructing a deep multi-layer neural network capable of inferring latent behavioral anomalies directly from raw socket connections. The system automatically processes network traffic parameters, minimizes manual feature engineering, and provides high-confidence predictive analytics before intrusions disrupt business continuity.

3. DATASET & PREPROCESSING

The development process utilized the benchmark **CICIDS2017 dataset**, which mimics real-world background activity and contains contemporary cyberattack signatures gathered by the Canadian Institute for Cybersecurity. The dataset covers deep execution traces for advanced threat classes including Heartbleed, Web Attacks, Infiltration, Botnets, PortScans, and multi-vector DDoS profiles.

Data Integrity Processing Metrics

The original multi-gigabyte capture was structurally audited to strip NaN artifacts, infinity values generated by division-by-zero flow parameters, and duplicated row vectors to guarantee mathematical convergence during backpropagation.

The operational preprocessing workflow is systematically outlined below:

- **Data Cleaning:** High-variance statistical features generating extreme non-finite anomalies (∞) were systematically imputed or dropped.
- **Categorical Encoding:** High-cardinality target classifications were map-transformed into clean ordinal representation vectors suitable for multi-class cross-entropy computation.
- **Feature Scaling:** Normalization was performed via standard statistical scaling where the normalized value is computed as:

$$z = (x - \mu) / \sigma$$

This stabilizes the gradients and guarantees accelerated stochastic descent convergence during PyTorch structural execution.

- **Data Partitioning:** The complete matrix was divided into structurally isolated training (70%), validation (15%), and testing (15%) subsets to prevent structural data leakage.

4. METHODOLOGY & SYSTEM ARCHITECTURE

The framework leverages an end-to-end multi-layered perceptron (MLP) built inside **PyTorch**, optimized with customized dropout regulations to mitigate overfitting. The core model extracts patterns from 78 network features to predict target classes accurately.

Workflow Pipeline Stage	Technology Applied	Functional Objectives & MLOps Purpose
Model Layer Architecture	PyTorch (Python)	Multi-layered neural network with ReLU activations and Softmax output.
Experiment Tracking	MLflow	Logs loss parameters, structural validation accuracy, and stores artifact binaries.
Application Layer	FastAPI	Exposes asynchronous REST API endpoints for high-throughput real-time inference.
Containerization	Docker	Encapsulates the runtime environment inside immutable, lightweight images.
Version Control & CI/CD	GitHub & Actions	Automates checking linting standards and executes build actions on repository changes.

Workflow Pipeline Stage	Technology Applied	Functional Objectives & MLOps Purpose
Orchestration	Kubernetes (Minikube)	Manages container scaling, self-healing nodes, and declarative traffic balancing.

5. EVALUATION & RESULTS

The model demonstrated exceptional generalization capabilities across unseen production traffic patterns. Optimization was driven by an Adam optimizer tracking standard multi-class cross-entropy loss metrics.

- **Classification Accuracy:** The system achieved highly precise detection rates for malicious behaviors while keeping false-positive indicators to a minimum.
- **Production API:** The FastAPI web engine successfully processes network feature vectors and delivers real-time classification inferences in milliseconds.
- **Scalable Infrastructure:** Docker containerization ensures identical performance profiles across development environments and production Kubernetes multi-node instances.

6. PROJECT DELIVERABLES

The technical components completed and delivered under this project scope encompass:

- **Deep Learning Model:** Highly structured PyTorch source implementation binaries.
- **FastAPI REST API:** High-performance request/response endpoints accepting tabular data payloads.
- **MLflow Tracking:** Centralized metrics server configurations detailing model lineage.
- **Docker Container:** Fully structured Dockerfile with minimized baseline layers.
- **CI/CD Workflow:** Declarative `.github/workflows` configuration file automating software pipelines.
- **Kubernetes Deployment:** YAML manifest profiles mapping high-availability Pod replicas.
- **Demonstration Video:** Real-time capture displaying end-to-end operation.

7. PROJECT REPOSITORY & DIGITAL ASSETS

All source code, configuration matrices, deployment scripts, and the detailed demonstration video have been indexed and uploaded to the centralized storage folder link provided below:

Google Drive Access Directory:

<https://drive.google.com/drive/folders/1mF0t1Sa0b9wxGVR1PfSh5WY7x7G6jQfm?usp=sharing>

CONCLUSION

By bridging deep learning architectures with modern cloud-native operational paradigms, this project showcases a comprehensive, enterprise-ready implementation of an AI-Powered Network Intrusion Detection System. Transitioning from mathematical models to production-grade Kubernetes orchestration workflows underscores the critical value of MLOps methodologies in securing modern digital systems against evolving cyber threats.